AGENT 1 - MONITORING
Rapport de travail

Ce qui a été construit, comment ça marche, et les résultats réels

Projet : Intelligent Multi-Agent System for Autonomous DevOps / MLOps Pipelines

Élément	Valeur
Responsable	Member 1 - Farah Marzougui
Composant	Agent 1 - Monitoring & Anomaly Detection (src/agents/monitoring/)
Dépôt / branche	Agent-1---Monitoring-&-Anomaly-Detection
État	Détection, regroupement, boucle autonome, raisonnement et décision faits ; 117 tests de l'agent (164 pour tout le projet) ; validé en test réel
Date	10 octobre 2026

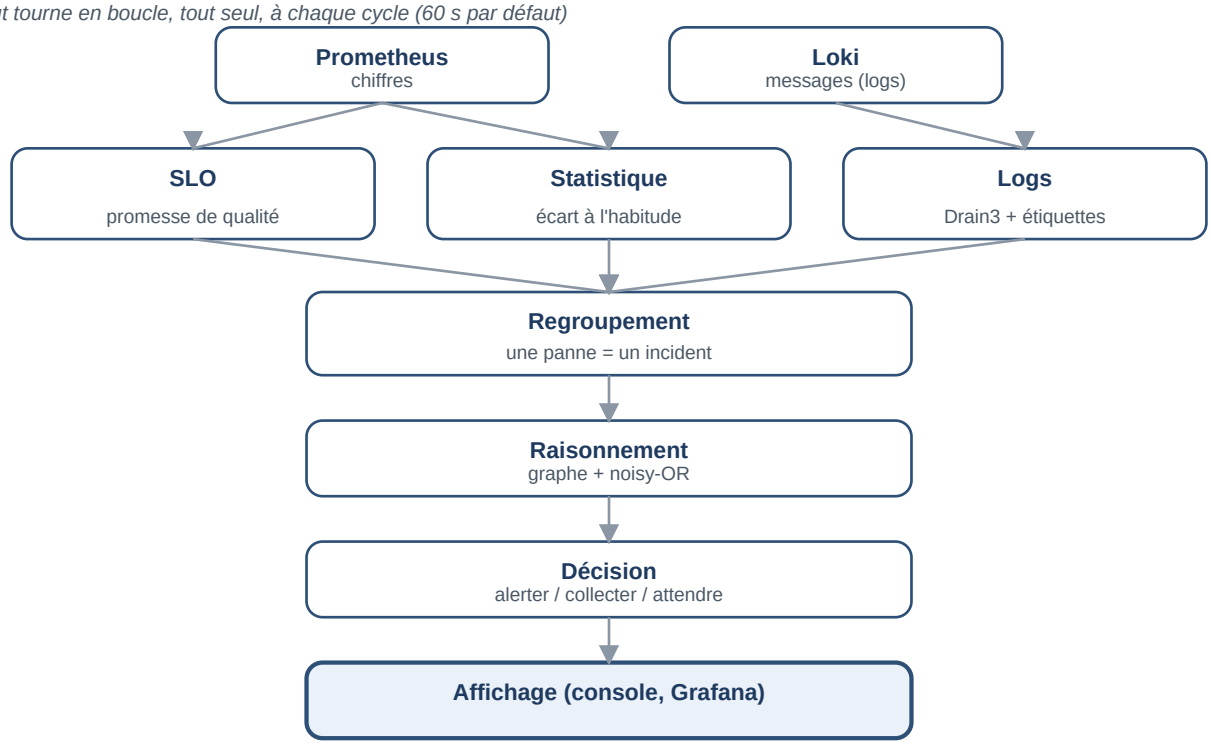
1. L'agent en une page

L'Agent 1 surveille les services, détecte les comportements anormaux et prévient la plateforme. Il rapporte des **faits observés** (quoi, où, quand, à quel point c'est grave), jamais la cause : le pourquoi appartient à l'Agent Diagnostic.

L'image de l'hôpital

Le service surveillé est le **patient**. Prometheus et Loki sont son **dossier médical** (chiffres et messages). L'agent est le **médecin de garde** : toutes les 60 secondes, il lit le dossier, vérifie avec trois méthodes, regroupe ce qu'il voit, réfléchit à l'origine du problème et décide s'il faut alerter.

Tout tourne en boucle, tout seul, à chaque cycle (60 s par défaut)



Couche	La question à laquelle elle répond
Collecteurs	Quels sont les chiffres et les messages du service ?
SLO	La promesse de qualité est-elle en danger ?
Statistique	Est-ce différent de d'habitude (signes avant-coureurs) ?
Logs	Quel type d'erreur apparaît ?
Regroupement	Est-ce une nouvelle panne ou la même ?
Raisonnement	Quel service est probablement à l'origine ?
Décision	Faut-il alerter, chercher plus d'indices, ou attendre ?

2. Le banc d'essai (le patient de test)

Rôle : disposer d'un service qu'on peut rendre malade volontairement, pour prouver que l'agent détecte des pannes connues à l'avance.

Comment ça marche :

- Un faux service de paiement (*payment-service*) qui publie ses chiffres sur `/metrics` et écrit ses messages d'erreur.
- Des boutons de panne `/chaos/*` : mémoire, erreurs (4 types), lenteur, CPU, et `reset` pour guérir. Actifs seulement en local.
- Prometheus, Loki, Promtail, cAdvisor et Grafana dans Docker, avec des limites de mémoire.
- Un générateur de visiteurs (5 requêtes par seconde).

Fichiers : `experiments/target_app/, experiments/load/traffic.py, compose.yaml`

Tests automatiques : pas de tests unitaires (c'est l'outil de test lui-même).

Exemple : injecter une panne

```
PS> Invoke-RestMethod -Method Post "http://127.0.0.1:8080/chaos/errors?rate=0.2&kind=timeout"
fault       rate kind
-----       ---- ----
http_errors  0.2 timeout
```

3. Les collecteurs (les yeux)

Rôle : lire de façon fiable les chiffres (Prometheus) et les messages (Loki) d'un service.

Comment ça marche :

- 6 signaux lus en parallèle : mémoire, pente mémoire, CPU, trafic, taux d'erreur, latence p95.
- Protection contre les pannes : délai maximal, nouvelles tentatives, disjoncteur.
- « Inconnu » s'écrit *None*, jamais zéro : l'agent ne confond pas « je ne sais pas » et « tout va bien ».
- Pour les logs : seulement les lignes d'erreur, avec « maintenant » et « d'habitude » lus séparément.

Fichiers : `collectors/prometheus.py, collectors/loki.py, collectors/http.py, src/platform/resilience.py`

Tests automatiques : 16 (collecteur 9 + résilience 7).

Exemple : le bulletin lu par l'agent pendant une panne (test réel du 6 octobre)

```
service_id             payment-service
memory_mb              216.40
memory_slope_mb_min    30.12      <- panne injectée : 30 MB/min
request_rate_rps       4.96
error_rate             0.203      <- panne injectée : 20 %
latency_p95_ms         86.75
missing                ()
source_available       True
```

4. La détection SLO (la promesse de qualité)

Rôle : savoir si les utilisateurs sont vraiment touchés.

Comment ça marche :

- Deux promesses : 99,5 % des visites sans erreur, 99,5 % en 500 ms ou moins.
- Le **budget d'erreur** est la part d'erreurs autorisée (0,5 %). Le **burn rate** est la vitesse à laquelle il est consommé.
- Alerte seulement si une durée longue **et** une durée courte dépassent le seuil (méthode du SRE Workbook de Google) : ni fausse alerte pour un pic, ni alerte pour un problème terminé.
- Réponses possibles : BREACH (alerte), OK, ou INSUFFICIENT_DATA (« je ne sais pas »).

Fichiers : `detectors/slo.py`

Réglages : configs/monitoring.yaml, section `slo`

Tests automatiques : 15.

Exemple : 20 % d'erreurs, le budget brûle 27 fois trop vite (la latence reste OK)

```
SLO  availability   fast-burn  burn long=16.30 court=27.29 seuil=14.4 -> BREACH
SLO  availability   slow-burn  burn long=16.30 court=16.30 seuil=6    -> BREACH
SLO  latency-500ms  fast-burn  burn long=0.00  court=0.00  seuil=14.4 -> OK
```

5. La détection statistique (les signes avant-coureurs)

Rôle : voir les problèmes avant que les utilisateurs souffrent, en comparant à l'habitude du service.

Comment ça marche :

- « D'habitude » = les 30 dernières minutes ; « maintenant » = les 5 dernières.
- Croissance : pente et régularité (R^2) de la mémoire.
- Changement de niveau : médiane et score z robuste pour la latence et le CPU. La médiane ignore un pic isolé.
- Un changement doit aussi être assez grand en valeur absolue (CPU +0,1 cœur, latence +50 ms).

Fichiers : `detectors/statistical.py`

Réglages : configs/monitoring.yaml, section `statistical`

Tests automatiques : 13.

Exemple : mémoire et latence anormales (487 ms, sous la limite SLO), CPU normal

```
Statistique  memory_mb       pente=30.00/min r2=1.00                    -> ANOMALY
Statistique  latency_p95_ms  actuel=487.50 habituel=89.40 z=89.06       -> ANOMALY
Statistique  cpu_cores       actuel=0.03 habituel=0.03 z=2.98           -> NORMAL
```

6. L'analyse des logs (le type d'erreur)

Rôle : résumer des milliers de messages et signaler les erreurs nouvelles ou en forte hausse.

Comment ça marche :

- Les secrets (mots de passe, jetons, emails, adresses IP) sont masqués en premier.
- **Drain3** regroupe les messages qui se ressemblent en quelques modèles.
- Une étiquette dans une liste fermée de 7 (CONNECTION_REFUSED, TIMEOUT...) : règles d'abord, **Gemini** en option seulement pour les messages inconnus.
- NEW_PATTERN : message jamais vu ; SURGE : message connu, 5 fois plus fréquent.

Fichiers : `detectors/logs.py, labelers/gemini.py`

Réglages : configs/monitoring.yaml, section `logs`

Tests automatiques : 25.

Exemple : un nouveau type d'erreur « Timeout », détecté en moins d'une minute

```
Logs  NEW_PATTERN TIMEOUT  recent=72 habituel=0  -> ANOMALY
>> INCIDENT OUVERT INC-2026-b21737634d payment-service network_errors gravite=MEDIUM sources=logs
```

7. Le regroupement (une panne = un incident)

Rôle : éviter des dizaines d'alertes pour une seule panne.

Comment ça marche :

- Signature = service + environnement + type de symptôme, sans l'heure.
- Même signature : l'incident est mis à jour ; signature nouvelle : nouvel incident.
- Notification seulement si l'incident est nouveau, plus grave, ou confirmé par une autre couche.
- Deux symptômes différents ne sont jamais fusionnés (ce serait déjà un diagnostic).

Fichiers : `grouping.py`

Tests automatiques : 12.

Exemple : la même panne vue 20 fois reste un seul incident ; sa gravité monte en HIGH

```
>> INCIDENT MIS A JOUR INC-2026-5586ae090c payment-service error_rate gravite=HIGH sources=slo vu 20 fois
```

8. La boucle autonome (le médecin travaille seul)

Rôle : faire de l'ensemble un vrai agent, qui surveille en continu sans intervention.

Comment ça marche :

- Un cycle toutes les 60 s : lire, détecter, regrouper, raisonner, décider, afficher.
- Une couche ou une source en panne ne bloque pas les autres ; chaque service a un délai maximal.
- Mode `--verbose` : affiche la décision de chaque couche, même quand tout va bien.
- Toute la configuration est dans `configs/monitoring.yaml`.

Fichiers : `loop.py, __main__.py, config.py`

Tests automatiques : 15.

Exemple : un cycle détaillé (légèreté : environ 1 s de calcul et 50 Mo)

```
PS> .\.venv\Scripts\python.exe -m src.agents.monitoring --verbose
Agent 1 demarre : payment-service, un cycle toutes les 60s, logs actifs, Gemini non configure, raisonnemen
[17:01:16] cycle 1.34s, memoire de l'agent 52 Mo, 1 service(s), 2 incident(s) observe(s) ce cycle
   payment-service (local)
     SLO          availability    fast-burn  burn long=16.30 court=27.29 seuil=14.4 -> BREACH
     Statistique  memory_mb       pente=0.00/min r2=0.00                           -> NORMAL
     Statistique  cpu_cores       actuel=0.02 habituel=0.02 z=0.17                 -> NORMAL
     Logs         NEW_PATTERN TIMEOUT              recent=72 habituel=0             -> ANOMALY
```

9. Le raisonnement (relier les problèmes)

Rôle : trouver quel service est probablement à l'origine des problèmes observés ensemble.

Comment ça marche :

- Un plan des dépendances (« payment-api dépend de payments-db ») dans `configs/dependencies.yaml`.
- Indices : symptôme sur le service, symptôme sur un service qui en dépend, message de log qui pointe vers une dépendance (« connection pool exhausted »).
- Combinaison par **noisy-OR** : P = 1 - (1 - w1)(1 - w2)...
- Résultat : des hypothèses classées, avec leurs preuves, transmises à l'Agent Diagnostic.

Fichiers : `reasoning.py`

Réglages : configs/dependencies.yaml

Tests automatiques : 9.

Exemple : la démonstration simulée (experiments/checks/demo_reasoning.py)

```
>> INCIDENT OUVERT ... payment-api  latency
>> INCIDENT OUVERT ... payment-api  log_errors
>> INCIDENT OUVERT ... payments-db  memory_pressure
?? HYPOTHESES (service a l'origine le plus probable) :
   1. payments-db  0.92  preuves : latence de payment-api (qui en depend) ; erreurs de payment-api ;
                         log "connection pool exhausted" ; memoire anormale de payments-db
   2. payment-api  0.58  preuves : latence ; erreurs
!! DECISION : ALERTER payments-db (probabilite 0.92) - assez sur de lui
```

10. La décision (choisir quoi faire)

Rôle : choisir entre alerter, attendre et collecter plus d'indices quand l'agent hésite.

Comment ça marche :

- Règles métier d'abord : un incident HIGH ou CRITICAL est toujours alerté.
- Sinon, un score par action (bénéfice moins coût) : ALERTER = p x 10 - (1 - p) x 6 ; ATTENDRE = -p x 4 ; COLLECTER = le meilleur des deux + 4 x doute - 2.
- Collecter = interroger Prometheus sur les suspects ; si c'est confirmé, la probabilité monte, sinon elle baisse.
- Pas de répétition de la même alerte ; mais un nouveau type de symptôme ou une forte hausse de probabilité alerte de nouveau.
- Aucun LLM dans la décision : elle doit être explicable et reproductible.

Fichiers : `decision.py`

Réglages : configs/monitoring.yaml, section `decision`

Tests automatiques : 12.

Exemple : l'agent hésite, vérifie, voit que tout est normal, donc il n'alerte pas

```
.. collecte sur payment-service : not_confirmed (error_rate=0.00, latency_p95_ms=89.64, memory_slope_mb_mi
!! DECISION : ATTENDRE payment-service (probabilite 0.25) - apres collecte : payment-service not_confirmed
```

11. Grafana (l'écran)

Rôle : voir les courbes du service pendant que l'agent travaille.

Comment ça marche :

- Tableau de bord chargé automatiquement : mémoire, taux d'erreur (ligne rouge à 0,5 %), latence p95 (ligne rouge à 500 ms), trafic, et messages d'erreur de Loki.
- Adresse : http://127.0.0.1:3000, lecture sans connexion en local.

Fichiers : `infrastructure/observability/dashboards/agent1-service-health.json`

Tests automatiques : aucun (affichage seulement).

12. Défauts trouvés grâce aux tests réels

Les tests réels ont révélé des défauts invisibles pour les tests automatiques. Chacun a été corrigé, et un test automatique le reproduit désormais pour qu'il ne revienne pas.

Défaut observé	Cause	Correction
Panne de 30 MB/min mesurée à 60	Arrondi dans l'application de test	Croissance calculée en cumulé
Un cycle durait 18,6 s pour 15 s d'intervalle	Vérification de Prometheus sans limite de temps	Vérification en parallèle, avec la même limite
Logs « NORMAL » malgré 175 erreurs	Plus de 10 000 lignes lues, coupées à 5000 : les erreurs récentes disparaissaient	Seulement les lignes d'erreur, deux fenêtres lues séparément
Fausse alerte CPU (0,00 vers 0,02 cœur)	Service au repos avant le test : « habitude » = 0	Changement minimum absolu (CPU +0,1, latence +50 ms)
Panne mémoire non signalée	Anti-répétition par service seulement	Nouvelle alerte si nouveau symptôme ou forte hausse de probabilité

13. Test réel final (après corrections)

Le test complet, sans aucune aide humaine : fonctionnement normal, panne mémoire, panne d'erreurs, puis guérison. Voici les moments clés tels que l'agent les a affichés.

Déroulé du test

```
== 17:52:20 PHASE 1 : fonctionnement normal
== 17:54:21 PHASE 2 : panne memoire +30 MB/min
== 17:59:23 PHASE 3 : panne erreurs 20% timeout
== 18:02:25 PHASE 4 : guerison
== 18:05:36 FIN
```

Phase	Résultat	Verdict
1. Normal	Le SLO lent se souvenait d'erreurs d'un test précédent. L'agent a hésité, vérifié (erreurs actuelles : 0 %), puis décidé d'ATTENDRE. Aucune fausse alerte CPU.	Correct
2. Mémoire (17:54:21)	17:55:52 : collecte « confirmed », ALERTER (probabilité 0,80). 17:57:22 : incident memory_pressure ouvert et nouvelle alerte « nouveau symptôme ». Pente mesurée jusqu'à 29,98 MB/min (R^2 = 1,00).	Correct, les deux corrections fonctionnent
3. Erreurs (17:59:23)	Pas de nouvelle alerte : le message « Timeout » avait été injecté 20 minutes plus tôt (test précédent), donc il n'était pas nouveau, et la hausse a duré trop peu pour atteindre le seuil « 5 fois l'habitude ». L'incident error_rate était déjà ouvert.	Non probant : à refaire sur un banc reposé
4. Guérison (18:02:25)	Docker Desktop s'est arrêté à 18:04 (fermeture normale, RAM libre très basse). L'agent a continué sans planter et a signalé les sources indisponibles.	Robustesse confirmée

17:52:22 - Phase 1 : il hésite, vérifie, et attend (pas de fausse alerte)

```
[17:52:22] cycle 0.58s, memoire de l'agent 52 Mo, 1 service(s), 1 incident(s) observe(s) ce cycle
     SLO          availability    slow-burn  burn long=16.03 court=15.13 seuil=6 -> BREACH
     Statistique  memory_mb       pente=0.00/min r2=0.00                          -> NORMAL
     Statistique  latency_p95_ms  actuel=- habituel=- z=-                         -> INSUFFICIENT_DATA
     Statistique  cpu_cores       actuel=0.00 habituel=0.01 z=-0.64               -> NORMAL
     Logs         aucun modele nouveau ou en hausse                                -> NORMAL
   >> INCIDENT OUVERT INC-2026-fcefb45b19 payment-service error_rate gravite=MEDIUM sources=slo vu 1 fois
   .. collecte sur payment-service : not_confirmed (error_rate=0.00, latency_p95_ms=47.50, memory_slope_mb
   !! DECISION : ATTENDRE payment-service (probabilite 0.25) - apres collecte : payment-service not_confir
```

17:55:52 - Phase 2 : la collecte confirme la panne mémoire, il alerte

```
[17:55:52] cycle 0.72s, memoire de l'agent 20 Mo, 1 service(s), 1 incident(s) observe(s) ce cycle
     SLO          availability    slow-burn  burn long=14.64 court=12.06 seuil=6 -> BREACH
     Statistique  memory_mb       pente=5.43/min r2=0.51                          -> NORMAL
     Statistique  latency_p95_ms  actuel=- habituel=- z=-                         -> INSUFFICIENT_DATA
     Statistique  cpu_cores       actuel=0.02 habituel=0.01 z=0.55                -> NORMAL
     Logs         aucun modele nouveau ou en hausse                                -> NORMAL
   .. collecte sur payment-service : confirmed (error_rate=0.00, latency_p95_ms=90.19, memory_slope_mb_min
   !! DECISION : ALERTER payment-service (probabilite 0.80) - apres collecte : payment-service confirmed
```

17:57:22 - Phase 2 : incident memory_pressure, nouvelle alerte « nouveau symptôme »

```
[17:57:22] cycle 0.59s, memoire de l'agent 19 Mo, 1 service(s), 2 incident(s) observe(s) ce cycle
     SLO          availability    slow-burn  burn long=14.09 court=8.70 seuil=6 -> BREACH
     Statistique  memory_mb       pente=18.24/min r2=0.86                         -> ANOMALY
     Statistique  latency_p95_ms  actuel=- habituel=- z=-                         -> INSUFFICIENT_DATA
     Statistique  cpu_cores       actuel=0.02 habituel=0.01 z=0.64                -> NORMAL
     Logs         aucun modele nouveau ou en hausse                                -> NORMAL
   >> INCIDENT OUVERT INC-2026-e7064ecb2f payment-service memory_pressure gravite=MEDIUM sources=statistic
   !! DECISION : ALERTER payment-service (probabilite 0.80) - assez sur de lui ; nouveau symptome : memory
```

17:59:22 - Phase 2 : la pente mesurée atteint 29,98 MB/min (panne : 30)

```
[17:59:22] cycle 3.09s, memoire de l'agent 17 Mo, 1 service(s), 2 incident(s) observe(s) ce cycle
     SLO          availability    slow-burn  burn long=13.42 court=6.91 seuil=6 -> BREACH
     Statistique  memory_mb       pente=29.98/min r2=1.00                         -> ANOMALY
     Statistique  latency_p95_ms  actuel=90.22 habituel=89.58 z=0.14              -> NORMAL
     Statistique  cpu_cores       actuel=0.02 habituel=0.02 z=0.16                -> NORMAL
     Logs         aucun modele nouveau ou en hausse                                -> NORMAL
   !! DECISION : ALERTER payment-service (probabilite 0.80) - assez sur de lui (deja alerte, situation inc
```

14. Tests automatiques

Partie	Fichier de test	Nombre
Collecteur Prometheus	test_monitoring_collectors.py	9
Résilience (retry, disjoncteur)	test_resilience.py	7
SLO	test_monitoring_slo.py	15
Statistique	test_monitoring_statistical.py	13
Logs (+ Gemini)	test_monitoring_logs.py	25
Regroupement	test_monitoring_grouping.py	12
Boucle autonome	test_monitoring_loop.py	15
Raisonnement	test_monitoring_reasoning.py	9
Décision	test_monitoring_decision.py	12
Total de l'agent		**117** (164 pour tout le projet)

Exemple : lancer les tests d'une partie

```
PS> .\.venv\Scripts\python.exe -m pytest tests/unit/test_monitoring_collectors.py -v
tests/unit/test_monitoring_collectors.py::test_collects_every_signal_into_a_typed_snapshot PASSED [ 11%]
tests/unit/test_monitoring_collectors.py::test_empty_or_nan_series_are_unknown_not_zero PASSED   [ 22%]
tests/unit/test_monitoring_collectors.py::test_transient_5xx_is_retried PASSED                   [ 33%]
tests/unit/test_monitoring_collectors.py::test_prometheus_down_degrades_without_crashing PASSED  [ 44%]
...
9 passed in 2.80s
```

15. Comment lancer le test réel

Les commandes

```
# 1. Le banc d'essai (Docker Desktop lancé, PowerShell normal)
cd C:\Users\marzo\Downloads\agent1-repo
docker compose --profile observability up -d target-app prometheus loki promtail grafana

# 2. Les visiteurs (terminal à laisser ouvert)
.\.venv\Scripts\python.exe experiments/load/traffic.py --rps 5

# 3. L'agent (terminal à laisser ouvert : c'est là qu'on lit le résultat)
.\.venv\Scripts\python.exe -m src.agents.monitoring --verbose

# 4. Après environ 20 minutes, une panne, puis la guérison
Invoke-RestMethod -Method Post "http://127.0.0.1:8080/chaos/errors?rate=0.2&kind=timeout"
Invoke-RestMethod -Method Post "http://127.0.0.1:8080/chaos/reset"

# 5. Tout arrêter
docker compose --profile observability down
```

16. Ce qui reste à faire

Priorité	Étape
Prochaine	Mémoire : l'agent apprend des retours « vraie / fausse alerte » (loi Beta)
Indispensable	Confiance et priorité P0 à P3 ; publication sur Kafka au format du contrat ; preuves dans le Context Service ; vérification de la guérison ; stockage Redis
Importante	Collecteur d'événements (crashs, déploiements) ; service gRPC ; évaluation chiffrée ; documentation
Bonus	Chronos-Bolt en comparaison ; actions en dry-run ; banc d'essai à plusieurs services

Limites honnêtes

Les seuils et les poids sont des valeurs de départ, à calibrer pendant l'évaluation. Le raisonnement par graphe n'a été démontré qu'en simulation, car le banc d'essai n'a qu'un service. Les incidents ouverts sont gardés en mémoire vive (Redis prévu).